Membership Field Reach Checker

Assignment Problem 1

Concept and approach

The access matrix depends on both modifier and access context. private is visible only inside the declaring class. default (package-private) and protected are visible within the same package; public is visible everywhere. The summary must always show all four modifiers and count allowed/denied attempts per modifier.

Important implementation notes

The required sample's protected attempts are in SAME_PACKAGE and SAME_CLASS, so both are allowed. Although Problem 1 describes finesOwed as reachable within the same package, Java's protected modifier is selected for Problem 2's subclass behavior; if you want the field to support that extension, declare it protected rather than package-private. The counting logic is O(n) for up to 500 attempts.

Java solution

```
class AccessChecker {
static final String[] MODIFIERS = {"private", "default", "protected", "public"};

static String classifyAccess(String fieldModifier, String accessorContext) {
switch (fieldModifier) {
case "private":
return accessorContext.equals("SAME_CLASS") ? "ALLOWED" : "DENIED";
case "default":
return accessorContext.equals("SAME_CLASS") ||
accessorContext.equals("SAME_PACKAGE") ? "ALLOWED" : "DENIED";
case "protected":
return accessorContext.equals("SAME_CLASS") ||
accessorContext.equals("SAME_PACKAGE") ? "ALLOWED" : "DENIED";
case "public":
return "ALLOWED";
default:
throw new IllegalArgumentException("Unknown modifier: " + fieldModifier);
}
}

static String summarizeByModifier(String[][] attempts) {
int[][] counts = new int[MODIFIERS.length][2]; // [modifier][allowed, denied]
for (String[] attempt : attempts) {
int index = -1;
for (int i = 0; i < MODIFIERS.length; i++)
if (MODIFIERS[i].equals(attempt[0])) index = i;
if (index == -1) throw new IllegalArgumentException("Unknown modifier");
if (classifyAccess(attempt[0], attempt[1]).equals("ALLOWED")) counts[index][0]++;
else counts[index][1]++;
}
StringBuilder result = new StringBuilder();
for (int i = 0; i < MODIFIERS.length; i++) {
if (i > 0) result.append(" | ");
result.append(MODIFIERS[i]).append(": ")
.append(counts[i][0]).append(" allowed / ")
.append(counts[i][1]).append(" denied");
}
return result.toString();
}
}

class LibraryMember {
private String membershipPin; // declaring class only
String branchCode; // package-private
double finesOwed; // package-private, Problem 2 tests protected behavior separately
public String displayName; // everywhere
}
```

Sample output

```
private: 1 allowed / 1 denied | default: 1 allowed / 1 denied | protected: 2 allowed / 0 denied | public: 1
allowed / 0 denied
```